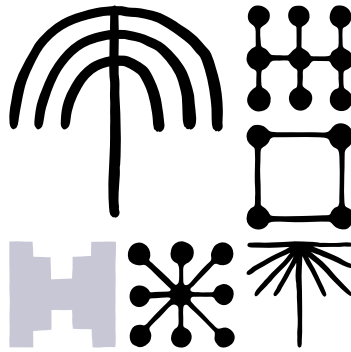


我们如何在各产品中限制 Claude 的影响范围

How we contain Claude across products

Anthropic · 2026-05-25 · 原文 / Source

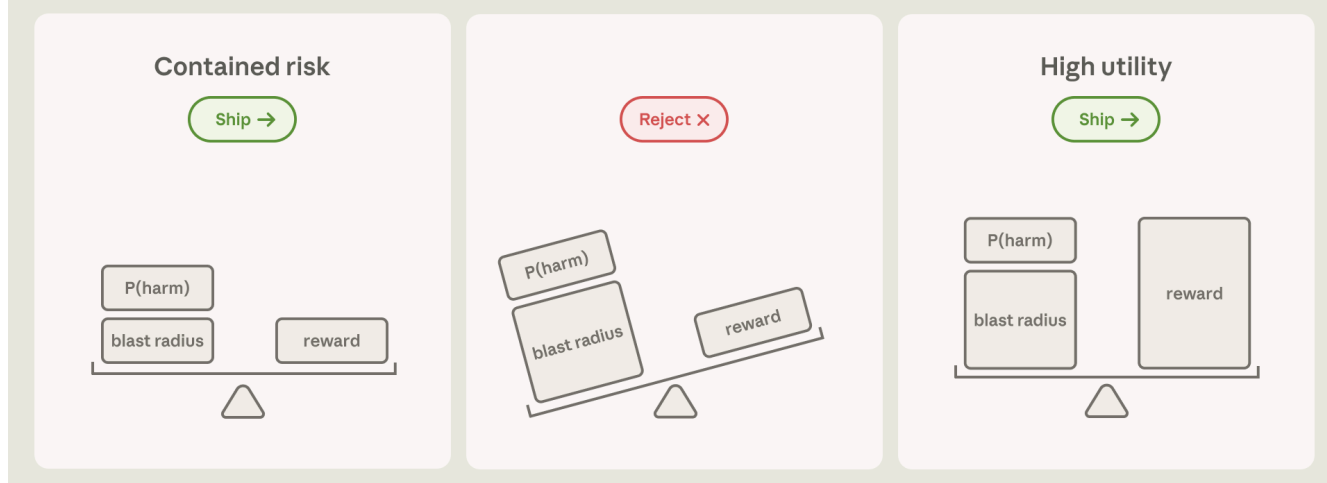
完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



随着 Agent 能力增强，其潜在影响范围也在扩大。工程上的问题，是如何为这一范围设定上限。以下是我们为 claude.ai、Claude Code 和 Cowork 构建隔离约束机制时积累的经验。

十二个月前，如果有人提议给予 Claude 足以让 Anthropic 内部服务瘫痪的访问权限，我们会毫不犹豫地拒绝。如今，这种级别的访问已是常态，Anthropic 开发者也因此提高了生产力。这些部署的风险包含两部分：故障发生的可能性，以及一次故障可能造成的损害程度。安全保障和模型训练方面的进步，持续降低了前者；而后者，即理论上的影响范围，只会随着能力和访问权限的扩展而增长。然而，当 Agent 能够完成过去需要一个人、甚至一个团队才能完成的工作时，不部署的代价也越来越高。只要产品能够做得足够安全，风险收益权衡就会明显倾向于采用。于是，工程问题变成了如何为影响范围设定上限。

Visualizing risk and reward



When bounds can be placed on the relative damage of an autonomous agent—such as through control over its environment—high-utility capabilities can motivate deployment. Claude Mythos Preview is an example of a model whose blast radius was deemed too high to ship in April 2026. However, we expect broader release of models with similar levels of capability to become appropriate as defenders harden critical systems and safeguards mature—even though some risk will always remain. Model capability is an important factor in the total risk of an agent's deployment.

大体上有两种做法。

第一种是让人参与流程，监督 Agent 的行为。此前，Claude Code 通过在每一步向用户请求许可，防止 Agent 采取非预期操作。理论上这行得通，但我们发现这种方法并不可靠。遥测数据显示，用户批准了大约 93% 的权限请求。用户看到的审批提示越多，对每条提示的关注就越少，久而久之，监督也会变得明显不够仔细。最近，我们构建了 Claude Code 自动模式，将更安全的审批自动化，以减轻审批疲劳。但漏洞仍然存在：任何概率性防御都有非零的漏检率。¹

第二种限制影响范围的方法，也是本文大部分内容的重点，是隔离约束（containment）。我们不是监督 Agent 做什么，而是借助沙箱、虚拟机和出站流量控制等机制强制执行访问边界，约束它能够做什么。这是 Anthropic 工程团队投入最多的领域，也是许多最出人意料的安全失效发生的地方。

过去两年，我们发布了三款主要的 Agent 产品：claude.ai、Claude Code 和 Claude Cowork。它们分别服务不同人群，因此需要不同的隔离约束架构。本文分享哪些设计经受住了考验、哪些出了问题，以及我们在这过程中学到的 Agent 安全经验。

三类风险，三个防御组成部分

Agent 的安全风险可以归为三类：

用户滥用或误用：用户出于恶意或疏忽，指示 Agent 做有害的事情。这涵盖了各种情况：要求 Agent 绕过自己觉得烦人的检查、运行自己不理解的破坏性命令，以及明确要求实施伤害。

模型不当行为：Agent 执行了无人要求的有害操作。随着模型改进，它们在大多数行为评测中变得更加对齐，但这并不意味着风险必然下降。能力较弱的模型更容易误读情境，犯下明显错误。能力较强的模型错误更少，但也更擅长找到通往目标的意外路径，往往会绕过那些没人想到要明确写下的限制。

在 Anthropic，我们见过 Claude 模型为了完成任务而“好心地”逃出沙箱，翻查 git 历史以寻找编程测试的答案，以及自发识别正在运行的基准测试，以便解密其标准答案。每个模型都会带来一组新能力，而这些能力有时会被用于出人意料的方向。

外部攻击者：Agent 通过工具、文件或网络访问等外部途径遭到攻击。这一类既包括提示注入，也包括针对 Agent 运行时、编排层或代理的传统攻击。

在构建隔离约束和防御系统时，我们会在三个主要组成部分上施加防护：

Agent 的运行环境。我们通过进程沙箱、虚拟机、文件系统边界和出站流量控制，限制 Agent 能在哪里、以何种方式行动。目标是为 Agent 可以触及的范围设定硬性边界。例如，只要凭据从未进入沙箱，就无法从中外传，无论诱因是用户指令、模型找到的“创意”路径，还是攻击者。

严密的边界也意味着可以适当放松监督。Claude Code 的参考开发容器正是为此而存在：让 Agent 可以在无人值守的情况下运行，无需逐项审批。

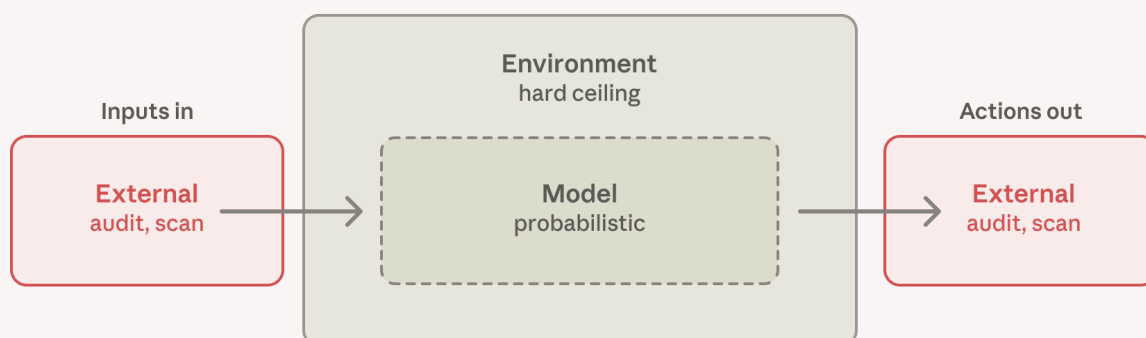
Agent 所调用的模型。这一层的机制包括系统提示词、分类器、探测器以及训练调整。由于模型具有概率性，这些措施只能塑造 Agent *倾向于*做什么，而不能决定它在理论上能够做什么。

这些防御很强。在测试提示注入易感性的 Gray Swan Agent Red Teaming 基准中，Claude Opus 4.7 将单次尝试的攻击成功率压到约 0.1%，在 100 次自适应尝试后，成功率约为 5% 至 6%。Claude Code 自动模式能在执行之前拦住约 83% 的过于积极行为。然而，即便采用一流防御，模型层的保护也永远不可能达到 100% 有效，这正是它不能独立承担全部防护责任的原因。

Agent 可以访问的外部内容。MCP 服务器、第三方插件和网页搜索工具，都会把你无法控制的来源中的内容送入 Agent 上下文。经过审计的连接，不等于经过审计的数据。例如，GitHub 连接器即使通过了恶意软件检查，也仍可能把一个被投毒的 README 直接加载进模型上下文。对工具权限进行细粒度限制，有助于限制影响范围。例如，只能读取数据库的 Agent，就比可以写入生产环境的 Agent 适合部署到广泛得多的场景中。

防御措施应当相互重叠、彼此补充。当环境层防御不可用时，模型层就必须补位，这正是 Claude Code 自动模式的设计用途。在本地，环境和模型防御可以抵御恶意工具输出；但也可以沿调用链向上游增加防护，限制工具本身的能力和访问权限。

Components to defending AI agents



Three components to defend: the model, the environment in which it runs, and the external content the agent can reach.

约束 Agent 的隔离模式

聚焦环境层，我们将介绍三种隔离模式，以及它们如何分别针对 claude.ai、Claude Code 和 Cowork 进行调整。每一种设计，都是在 Agent 所需能力与用户必须介入的程度之间逐步寻找平衡后形成的。

模式 1：临时容器（claude.ai 代码执行）

虽然 claude.ai 最为人熟知的是聊天界面，但它也会编写和运行代码、生成文件，以及调用连接器。当 Claude 在 claude.ai 中运行代码时，它使用的是隔离基础设施上的 gVisor 容器。Agent 完全运行在服务端，本地机器不会执行任何代码，文件系统也是临时的，按会话存在。潜在影响范围很小，但 Claude 的能力上限同样受限：没有持久工作区，也无法访问用户的文件系统。

这也意味着 claude.ai 面对的是更传统的威胁模型。我们不是要保护用户机器免受 Agent 影响，而是要保护自己的基础设施，并防止各租户相互影响。claude.ai 上线前的大部分工作，都是网络配置、内部服务身份验证和编排等传统安全工作。

这项工作再次印证了安全领域最古老的一条经验：最薄弱的一层，往往是你自己构建的那层。gVisor 和 seccomp 对抗资源充足攻击者的加固历史，比 Agent 型 AI 的存在时间长得多，因此我们的审查重点放在围绕它们构建的新组件上。后文会再次谈到这一点，因为在我们影响最重大的事件中，出问题的也正是自定义代理。

模式 2：人工参与的沙箱（Claude Code）

Claude Code 运行在用户机器上，可以访问用户的文件系统、Shell 和网络。缺少这些能力，编程 Agent 的用处就很有限，因此必须找到安全授予这些访问权限的方法。

一种办法是依靠人工参与监督。之所以这种方案对 Claude Code 可行，是因为它的典型用户是熟悉编程环境的开发者：能够读懂 bash，知道 `rm -rf` 会做什么，也已经习惯每周多次从不可信来源运行 `npm install`。这意味着，当“是否允许此操作”的对话框弹出时，他们很可能具备所需知识，能够准确评估 Agent 正在尝试做什么，以及相关风险。基于这一点，Claude Code 发布时采用了尽可能简单的防御：允许读取，写入、bash 和网络访问则需要批准。

然而，如前所述，审批疲劳在几周内就出现了。讽刺的是，这意味着一个本来用于提供监督的功能，可能反而产生相反效果：一些用户可能干脆不再留意。为了减少草率批准，我们首先发布了操作系统级沙箱（macOS 上使用 Seatbelt，Linux 上使用 bubblewrap），以强化边界：允许读取，允许在工作区内写入，但默认禁止网络访问。在沙箱内部，Agent 基本上可以不受打断地运行。结果是权限提示减少了 84%；我们还开源了运行时，让这一边界可以接受审计。

我们的匿名化使用数据还显示，有经验用户自动批准操作的频率约为新用户的两倍，但他们也更常在执行过程中打断 Agent。与其逐步把关，有经验的用户更倾向于只在 Agent 偏离方向时介入监督。这也许是人们与 Agent 协作偏好的一种自然演变，但同样并不可靠，因为首先就要求用户有足够技术能力和注意力，能够察觉偏离。随着模型能力提升、Agent 开始编写越来越复杂大胆的 bash 命令，察觉这种偏离会更加困难。而当用户转向多 Agent 系统时，这种方式作为监督策略有效的可能性也会大幅下降。

我们遗漏的风险：信任对话框出现之前的一切

从 2025 年年中到 2026 年 1 月，我们通过负责任披露计划收到了 Claude Code 的漏洞报告。其中三个漏洞针对的都是在用户尚未同意任何事项之前就会执行的代码。要理解这是如何发生的，可以看最直接的案例：开发者为了审查拉取请求而克隆了一个仓库，该仓库包含一个定义了钩子的 `.claude/settings.json`。由于 Claude Code 会在启动时读取项目设置，也就是在显示标准的“你信任此文件夹吗？”提示之前，攻击者编写并提交的钩子就会自动执行。其余案例在结构上类似：尚未建立信任边界，就已经解析了来自尚未获信任目录的输入。

每个案例的修复方式都相同：将项目本地配置的解析和执行延后，直到用户接受信任提示。如果你正在构建类似产品，应当像对待任何来自互联网的入站请求一样，对待项目打开、配置加载和 localhost 监听服务。不能因为它们给人“就在本地”的感觉，而且在用户同意之前就已到达，就默认信任它们。

我们遗漏的风险：用户本身成为注入途径

2026 年 2 月，在一次受控的内部红队演练中，一位研究人员成功通过网络钓鱼，诱使一名员工用恶意提示启动 Claude Code。这封钓鱼邮件看起来像普通协作：邮件写着“能帮我运行一下这个吗？”，并附上可直接粘贴的提示；提示本身看起来也是日常任务指令。但在某个准备步骤中，它不露声色地要求 Claude 读取 `~/.aws/credentials`，对内容编码，并将其 POST 到外部端点。对这条提示进行 25 次重复尝试，Claude 有 24 次完成了数据外传。

这是一次直接提示注入：攻击者的指令通过用户传入，而不是通过工具输出或抓取内容传入。我们的模型层防御以用户意图为依据；当指令由用户本人输入时，分类器就没有异常可抓。如果把同样的脚本交给一位人类外包人员，对方也会做同样的事。

这种情况下，唯一有效的防御来自环境：具体而言，是不论意图如何都阻止该 POST 请求的出站流量控制，以及从一开始就让 `~/.aws` 无法访问的文件系统边界。

(当我们在内部 Slack 中分享这条有效的提示进行讨论时，有人指出，一些内部 Agent 会读取 Slack。于是，这个载荷已经散布在环境中。我们在讨论串里加入了一段金丝雀字符串，以便在任何系统接触它时察觉。在一个 Agent 会读取一切的世界里，调查工具本身也是攻击面。)

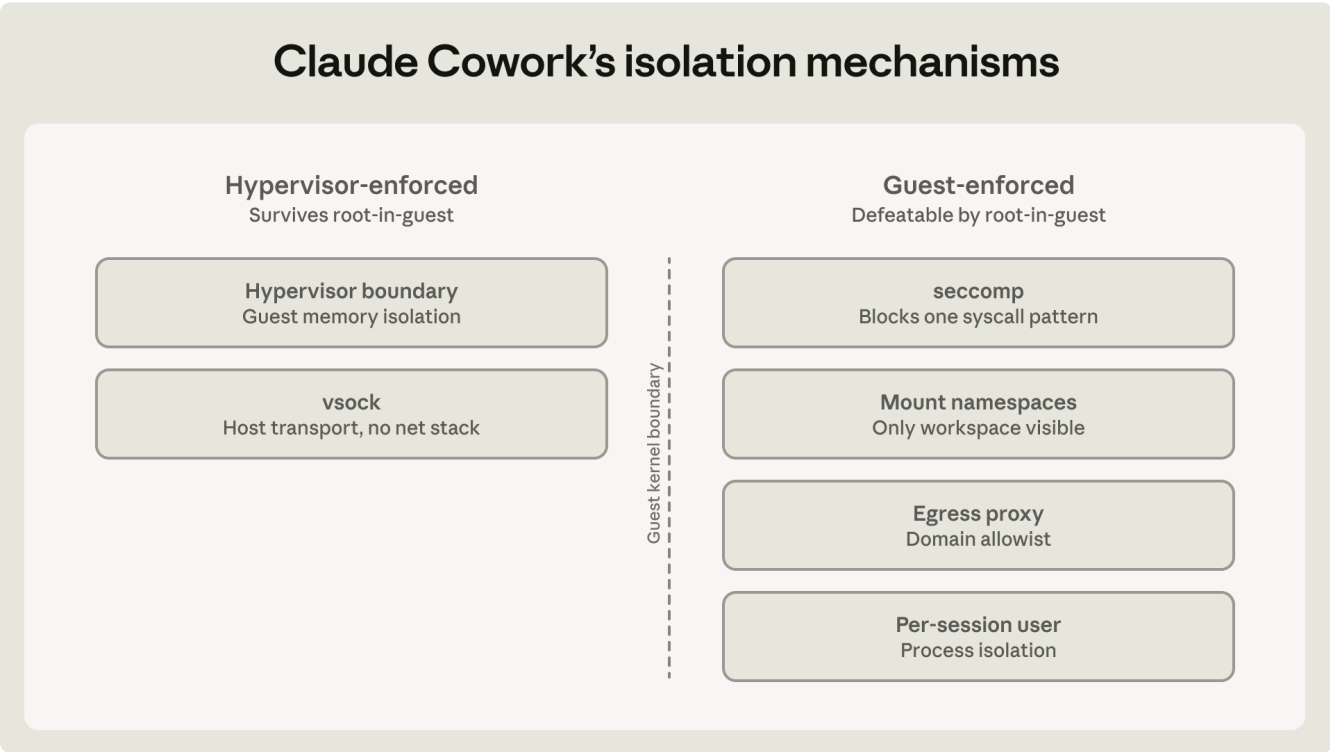
模式 3：本地虚拟机（Claude Cowork）

Claude Cowork 运行在用户桌面端，可以访问用户选择的工作区文件夹。由于该平台面向一般知识工作，而不是软件工程，典型用户熟练掌握 bash 的可能性要低得多。

因此，人工参与的沙箱策略未必能直接迁移；我们不应要求非技术知识工作者去判断 `find . -name "*.tmp" -exec rm {} \;` 这样的 bash 命令。当批准例外所需的专业知识超出典型用户的能力时，管理员就应设置一个绝对且始终生效的边界。

为此，第一版 Claude Cowork 运行在完整虚拟机中，使用平台厂商提供的虚拟机管理程序（macOS 上使用 Apple 的 Virtualization 框架，Windows 上使用 HCS）。虚拟机拥有自己的 Linux 内核、文件系统和进程表。用户选定的工作区和 .claude 文件夹会被挂载，宿主机上的其他内容都不可见。凭据保留在宿主机钥匙串中，从不进入来宾虚拟机。这种设计用于防范 Claude 在某个时刻表现出不对齐行为的可能性。遭到攻破的 Claude 仍然可能损坏工作区文件夹内的内容，因此架构的设计目标是：确保这就是它唯一能够触及的范围（直到用户添加连接器），并且由用户决定挂载哪些内容。

在最初的架构中，也就是我们所说的全虚拟机模式，Agent 循环本身也运行在来宾系统内，因此 Claude 以普通 Linux 用户身份执行，完全不知道自己处于沙箱中。相比之下，Claude Code 有一个位于沙箱外的特权进程，逐条命令决定是否施加沙箱约束；一条颇具说服力的注入提示，或一次因疲劳而草率点击的批准，都可能让该进程在沙箱外运行某些操作。而在这里，没有外层进程握着逃生通道的钥匙，因此也没有任何组件有权批准例外。

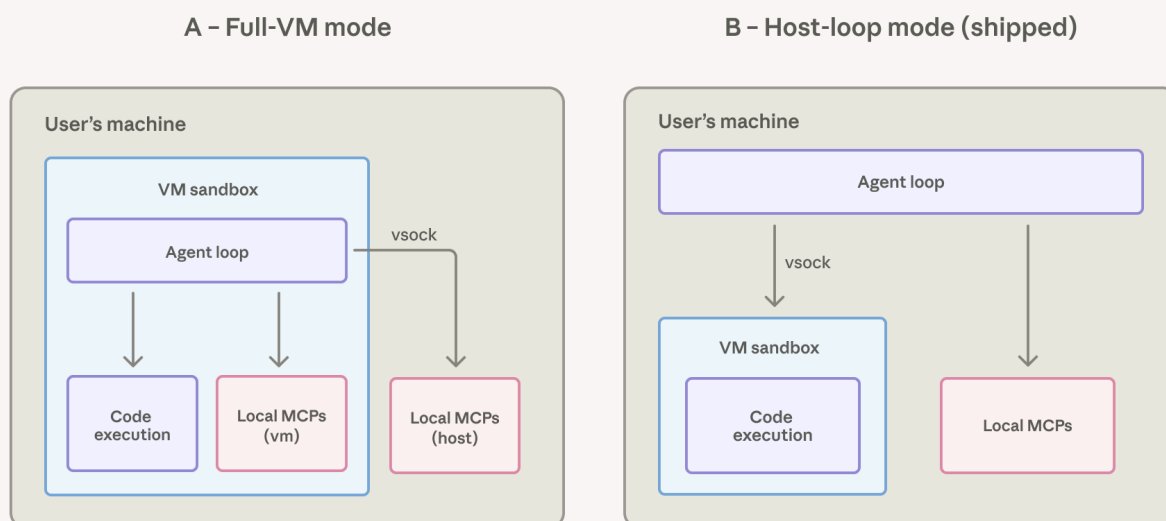


The six main isolation mechanisms of Claude Cowork's VM. Two are enforced outside the guest kernel and would thus survive the agent achieving root-level access within the VM. The other four are guest-enforced and kept deliberately minimal because the outer layers carry the rest.

然而，我们很快意识到，将整个 Agent 放进全虚拟机模式，会造成实际问题：虚拟机启动期间的任何故障，都会让 Cowork 无法使用。将 Agent 循环移到虚拟机外部，同时把代码执行保留在内部，就能让 Claude 仍然回应用户并协助调试，而不是遇到错误就卡住。这个变化对安全的影响很小，因为虚拟机仍然对 Agent 执行的代码强制实施文件系统和网络控制。

另外，我们也将本地 MCP 服务器移到了虚拟机外部。把它们运行在虚拟机内部，会使审计更困难，虚拟机更新时容易出现脆弱的依赖问题，而且无法支持需要与数据库等本地进程交互的 MCP——这类服务器无论如何都必须运行在宿主机上。这项调整让 Claude Cowork 与 Claude Desktop 现有的本地 MCP 服务器处理方式一致：将它们视为用户可能选择安装的普通软件，并由管理员决定启用哪些本地 MCP，或者一个也不启用。远程 MCP 服务器不受影响，因为它们不运行在用户机器上。

Comparison: Full-VM vs host-loop



Having the agent loop inside the VM meant that any failure in the VM caused Cowork to become unusable. Host-mode is more reliable because the agent can still respond if the VM crashes, and it still provides important security guarantees by isolating code execution.

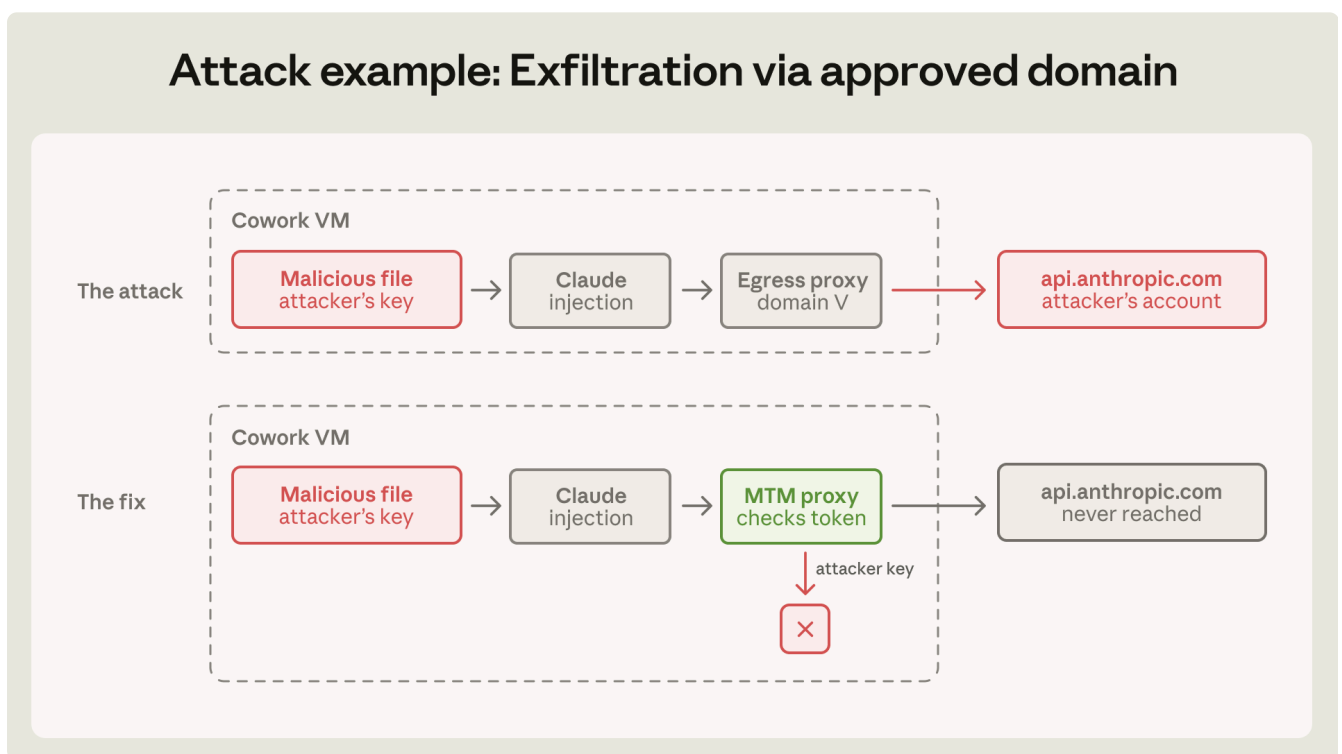
文件系统控制是另一项重要的架构选择。Claude 要发挥作用，就必须能够访问宿主机上的某些文件，但我们希望尽量缩小影响范围，并让用户清楚了解本地文件访问情况。我们发现，提供不同文件挂载模式有助于精细控制风险；Claude Cowork 提供只读、读写，以及可读写但不可删除三种模式。这里有一个潜在陷阱：必须在路径验证之前解析符号链接，而不是之后，否则授权文件夹中的符号链接可能指向外部，逃出边界。对于企业客户，我们允许管理员通过 MDM 设置中的挂载路径允许列表进行控制。

我们遗漏的风险：通过获准域名外传数据

一个通过获准域名外传数据的明确案例，来自第三方披露。Claude Cowork 的出站流量允许列表正确地放行了前往 `api.anthropic.com` 的流量，因为产品如果无法调用我们自己的 API，就不能正常工作。在这个案例中，用户挂载工作区内的一个恶意文件，携带了隐藏指令和攻击者控制的 API 密钥。Claude 遵循这些指令，读取工作区中的其他文件，并使用攻击者的密钥调用 Anthropic Files API。出站代理检查目标地址，看到 `api.anthropic.com`，就予以放行。于是，文件被上传到了攻击者的 Anthropic 账户。沙箱完美地完成了本职工作，但数据仍然被外传了。

此前，我们将允许列表理解为目标地址过滤器，也就是告诉 Claude：这些域名可以通信。但更合适的理解方式，可能是将它视为一种能力授权。允许列表中任意域名所能提供的每项功能，现在都成了攻击面。允许 `api.anthropic.com`，就意味着允许向任意 Anthropic 账户上传文件。

我们在虚拟机内部使用一个防御性的中间人代理，拦截发往我们 API 的流量，以此修复问题。它只放行携带该虚拟机自身已配置会话令牌请求；攻击者嵌入的密钥会被代理拒绝。它还会阻止那些能够启用服务端抓取请求头。代理之所以部署在虚拟机内，而不是我们的服务器上，是因为只有虚拟机知道请求来源；从服务器的角度看，Cowork 请求与任何其他 API 客户端请求并无区别。



Top: traffic to `api.anthropic.com` is let through, resulting in egress. Bottom: fix with a man-in-the-middle proxy intercepting traffic to our API.

这也是“自己编写的软件往往最薄弱”这一原则的第二个例子。我们各产品中的虚拟机管理程序、`seccomp` 和 `gVisor` 一直可靠。失效的恰恰是我们自定义的允许列表代理。

我们遗漏的风险：虚拟机隔离也把端点检测软件挡在了外面

企业安全团队在评估 Claude Cowork 时问道：“为什么我们的 EDR 看不到里面？”答案是，将 Claude 约束在内部的同一层隔离，也把基于宿主机的端点检测与响应系统挡在了外面。从 EDR 的角度看，Claude Cowork 是一个不透明的虚拟机管理程序进程，它无法检查来宾系统。

隔离降低了可见性，而对于合规工作依赖端点可见性的团队来说，这种不透明会带来问题。我们目前的缓解措施是使用基于拉取的 OTLP 导出，让管理员可以事后获取事件日志，但这与实时监控并不相同。如果你正在构建类似产品，应尽早为这类讨论预留时间和资源。

Environment	Ephemeral container (claude.ai)	HITL sandbox (Claude Code)	Sealed VM (Claude Cowork)
Cost: Isolation Overhead	Container spin-up	Low-latency native sandbox	Full VM boot
Cost: User Reliance	N/A	Must interpret bash	N/A
Risk: Blast Radius	Server-side container (guarded by gVisor + host infra boundary)	Local workspace	Mounted workspace (guarded by vsock + hypervisor boundary)

如何信任 Agent 读取的内容

企业经常问我们如何保障 MCP 连接安全。这是个好问题，但正确的问题范围比 MCP 本身更广。向 Agent 提供的任何外部资源，都同时带来两种风险：传统供应链意义上的代码执行风险，以及提示注入途径。传统依赖审计，例如锁定版本、验证签名和审查源码，能够应对第一种，却无法覆盖第二种。

远程与本地的区别，比看起来更重要。本地安装的工具可以审计。你可以阅读代码、锁定版本，并确定它不会在你不知情时变化。而远程工具，无论是托管的 MCP 服务器还是云连接器，都可能在你批准后的任何时刻改变行为；安装时作出的信任判断，可能已经不再适用。我们的连接器目录通过持续审查应对这一问题，但目录之外的任何工具都应被视为不可信。先用虚假数据测试，并将其放在一个能够限制恶意工具影响范围的环境中运行。

即便工具可信，工具输出也仍是攻击面。前文提到的 GitHub README 示例正是这种情况；任何用于网页的输入扫描，都应同样严格的标准应用于联网工具的返回结果。虽然这会增加延迟，也并不是完美防御，但我们倾向于实时检查：一旦遭到投毒的工具返回内容诱导 Agent 外传了数据，日志中只会显示一次成功且经过授权的 API 调用。事后并没有异常信号可供寻找。

在 Claude Code 和 Claude Cowork 中，工具调用经过代理路由；代理强制执行网络 and 文件策略，并且可以在返回值进入模型上下文之前进行检查。负责检查的分类器可以是一个小而快的模型，无需与执行推理的模型相同。

展望未来

模型和产品正在快速发展。与此同时，风险也在变形、演进，我们的缓解措施必须跟上步伐。

持久记忆投毒。Agent 上下文中会跨会话保留的部分正在不断增多，包括产品记忆、CLAUDE.md 文件、挂载工作区，以及定时运行和长时间运行 Agent 的状态目录。落入其中任何位置的注入，都会在 Agent 每次启动时重新加载。随着越来越多的 Agent 状态在会话结束后得以保留，我们面临着新的持久化机制威胁，类似于传统入侵后的持久驻留。会话启动时使用优秀分类器进行检查，需要变得更加普遍。

多 Agent 信任升级。一方面，子 Agent 可以隔离不可信内容，向主 Agent 返回结构化事实，而不是原始文本。另一方面，这也可能被滥用：如果仅仅因为输出来自“自己人”，就把子 Agent 的输出视为比原始工具结果更可信，那么就引入了一条新的提示注入途径。在多 Agent 系统中，为内容分配不同信任级别，与暴露于信任升级风险之间，存在取舍。

Agent 身份。Claude Cowork 对 Agent 身份问题给出了具体方案：凭据留在宿主机钥匙串中，虚拟机获得按会话发放、权限范围缩小的令牌，而且这个令牌可以独立于用户令牌撤销。但我们也正开始面对跨平台 Agent 身份这一更广泛的问题。Agent 应该拥有自己独立的安全主体身份，还是应作为用户的延伸行事，继承用户权限？最终答案可能是二者的结合。

随着 Agent 能力增强，攻击面也在持续变化。我们见过的这些失效类型，很可能在各行业和实验室中反复出现。我们需要共同投入，建立面向 Agent 的安全防护体系，从共享基准和披露规范，到通用身份标准与跨厂商红队测试。本文聚焦隔离约束，但它只是 Agent 安全全貌中的一部分。关于治理、可观测性和其余技术栈，可参阅 NIST 的 AI Agent 身份与授权项目、由澳大利亚 ACSC 牵头并由 CISA 和英国 NCSC 等参与的六机构 Agent 型 AI 采用指南，以及 AI 管理标准 ISO/IEC 42001。我们的 Glasswing 计划是其中一份贡献，但我们期待与合作伙伴和竞争对手共同应对这一关键问题。

总结

简而言之，有几条原则是我们反复回归的：

先和环境层设计隔离约束，再在模型层引导行为。对我们启发最大的两个事件——员工遭遇钓鱼，以及第三方披露的允许列表问题——都属于出站事件，数据都是通过获准路径流出的。在这两个事件中，模型层都无能为力，因为没有可供它识别的异常。当所有概率性措施都漏检时，最终承受冲击的就是确定性的边界。

让隔离强度与用户的监督能力相匹配。能够读懂 bash 的开发者，与不能读懂 bash 的知识工作者，面对的并不是同一种威胁模型。用户是否能够评估 Agent 即将执行的操作，应当帮助决定隔离约束策略。无论朝哪个方向判断错了——给专家制造过多阻力，或对非专家给予过多信任——本身都是一种失败。

警惕自定义组件。经过实战检验的虚拟机管理程序、系统调用过滤器和容器运行时，承受过的对抗性考验，比你构建的任何东西都多。在本文描述的每一种部署中，标准基础组件都经受住了考验，而我们围绕它们编写的部分却暴露出了缺陷。

归根结底，Agent 也许是一类新软件，但它们在系统层面的交互并不新。它们依然读取文件、打开套接字、创建进程，因此借助成熟工具实施隔离约束，是一种至关重要且切实可行的防御。随着 AI 发展，部署的风险收益平衡会继续变化；但为影响范围设定硬性上限，往往能够让这一平衡朝正确方向倾斜。

致谢

作者为 Max McGuinness、Mikaela Grace、Jiri De Jonghe、Jake Eaton 和 Abel Ribbink。

我们也感谢 Hanah Ho、Hasnain Lakhani、Pedram Navid、Molly Villagra、Maya Nielan、Akila Srinivasan、Travis Szucs、Sam Attard、Alfred Xing、Mohamad El Hajj、Gabby Curtis、David Dworken、Adam Jones、Amie Rotherham、Christian Ryan、Lucas Smedley、Brett Andrews 以及其他人的贡献。

特别感谢我们的安全与产品工程团队，以及向我们报告 Claude 产品漏洞的个人和组织。

脚注

1. Claude Code 自动模式将命令审批委托给基于模型的分​​类器；它将操作阻力降至很低（大约 0.4% 的无害命令会被拦截），代价是漏过一部分危险命令（约 17% 的过于积极操作会被放行）。因此，它是在沙箱内部实现纵深防御的一层，而不能替代沙箱。